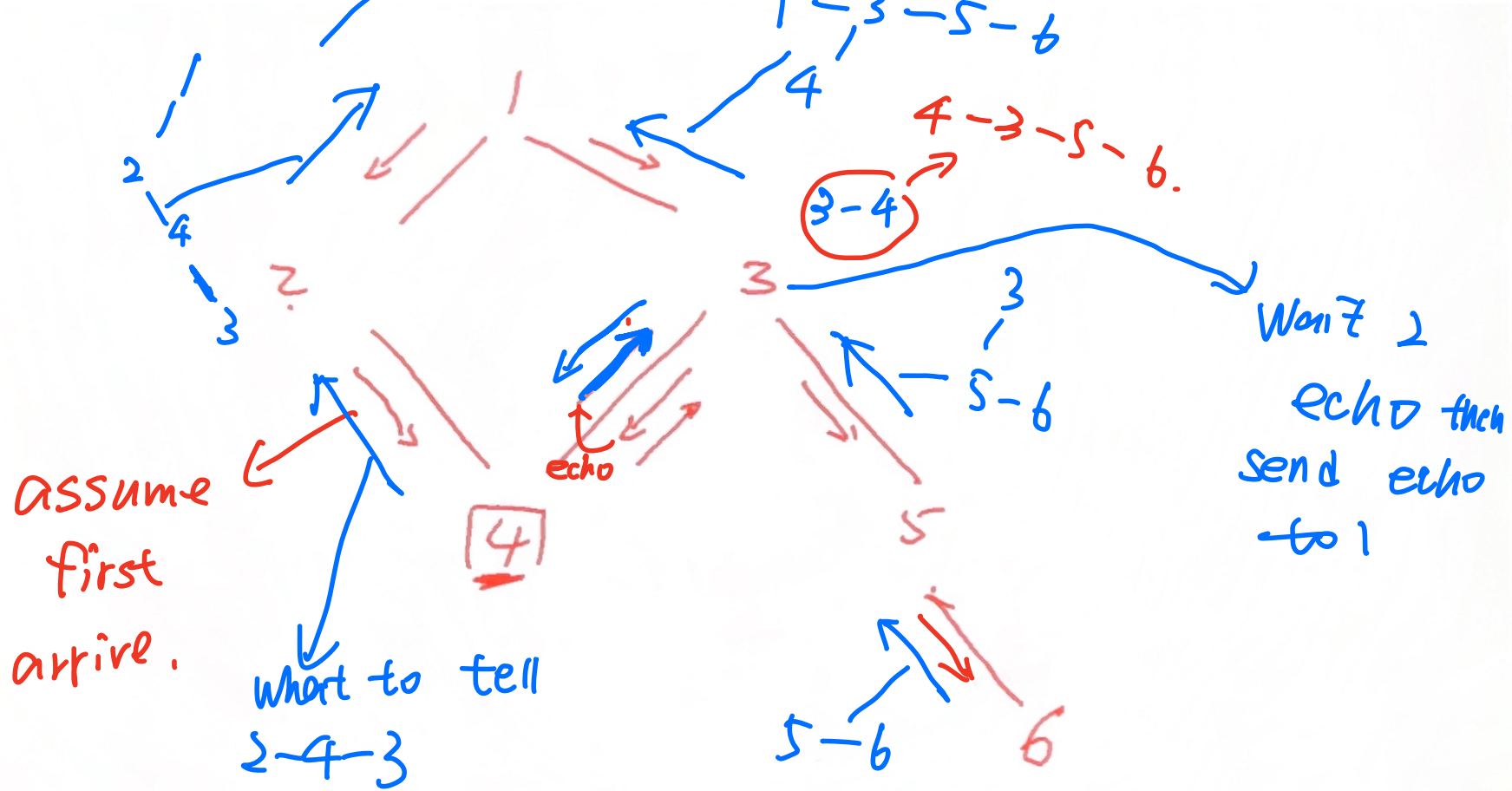


The Echo Algorithm

- The echo algorithm can be used to collect and disperse information in a distributed system
respond.
分散
- It was originally designed for learning network topology. But, it can be applied for solving many other problems
拓扑
- Each node only knows and communicates with its neighbours.





- ❑ The initiator starts the algorithm by **sending probes** to all its neighbours.
- ❑ The algorithm **terminates** when the initiator **receives echoes** from all its neighbours.
- ❑ A non-initiator node **replicates and sends probes** to its neighbours after receiving a probe.
- ❑ A non-initiator node **sends an echo** to the node from which it receives the probe when the node has received echoes from all its neighbours.

Deadlock detection in distributed systems

System model

- ❑ A computation consists of **a set of processes** running on a collection of processors connected by a communication network
- ❑ Without loss of generality, it is assumed that each process runs on a **different processor**
- ❑ The processors do not share a common global memory and communicate solely **by passing messages** over the communication network
- ❑ Processes make **exclusive access to resources** and there is only one copy for each resource
- ❑ A process can be in two states, **running or blocked**
 - A blocked process is waiting to acquire some resources

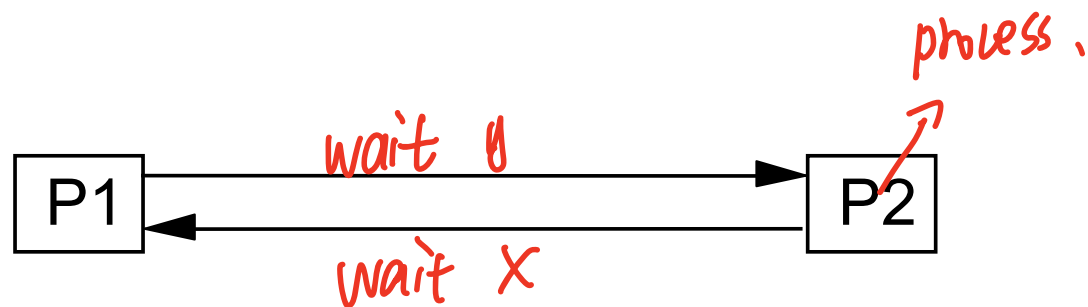
Wait-for graph

- A wait-for graph is a **directed** graph
- In a wait-for graph,
 - **nodes** are processes
 - There is a directed **edge** from node P1 to P2 if P1 is blocked and **waiting** for P2 to release some resources

□ Example *resource.*

○ P1: $w1[x]$ $w1[y]$ $r1[x]$

○ P2: $r2[y]$ $r2[x]$ $w2[y]$



*P1 access x → blocked.
P2 access y*

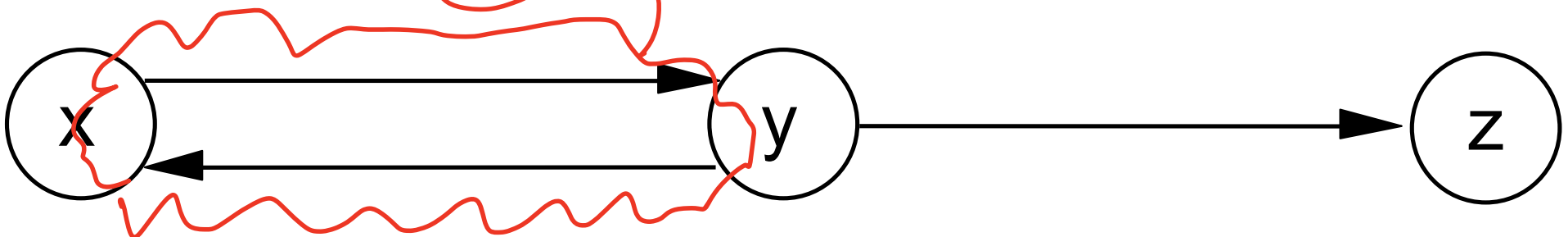
Models of Deadlock

□ There are two deadlock models

- AND model (resource model)
- OR model (communication model)

□ In the AND model

- A process is permitted to request a **set** of resources.
- The process remains **blocked** until it is granted **all** the resources it has requested.
 - Therefore, requests of this type are called AND requests.
- To find a deadlock in the **AND** model it is necessary to find a **cycle** in the wait-for graph.

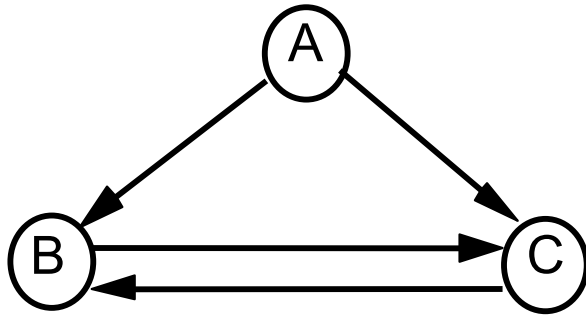


□ In the OR model

- A process can communicate with a **set** of other processes.
- The process can make progress as long as **one** of the communication occurs.
 - Therefore, in this model a **cycle** in the wait-for graph does not necessarily mean a deadlock exists
- To detect a deadlock, it is necessary to detect a knot in the wait-for graph.
 - All the processes being waited by other processes are in a cycle.

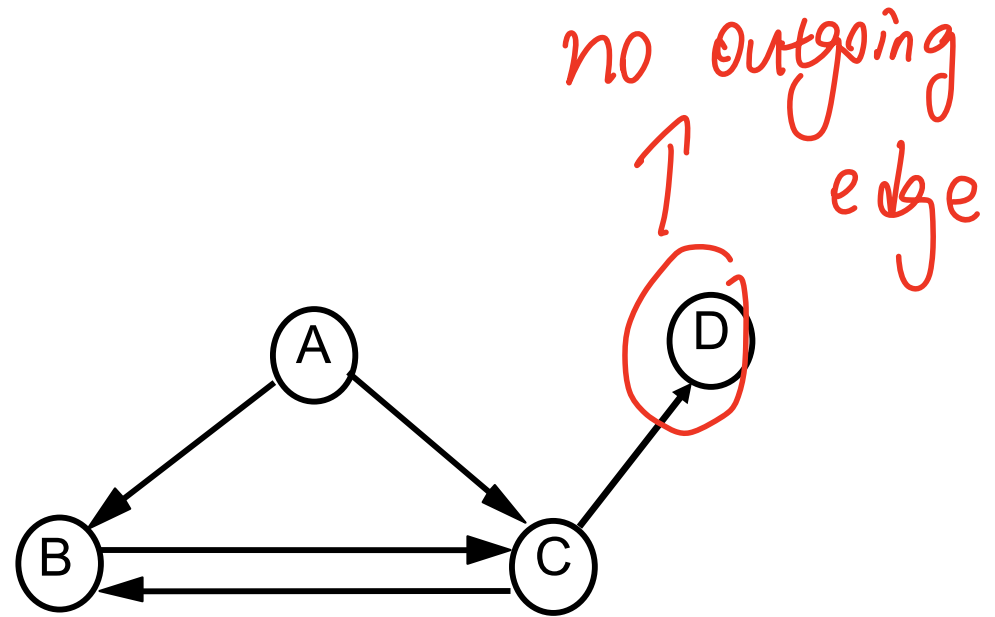
结 A **knot** in a directed graph is a collection of vertices and edges with the property that every vertex in the knot has **outgoing edges**, and all outgoing edges from vertices in the knot **terminate** at other vertices in the knot. Thus, it is impossible to leave the knot while following the directions of the edges. **everyone waiting for someone**

or model



① outgoing edge

② end with node
in the same set.



no outgoing
edge

Deadlock Detection

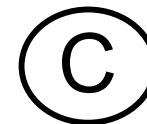
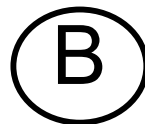
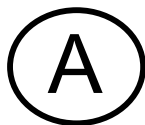
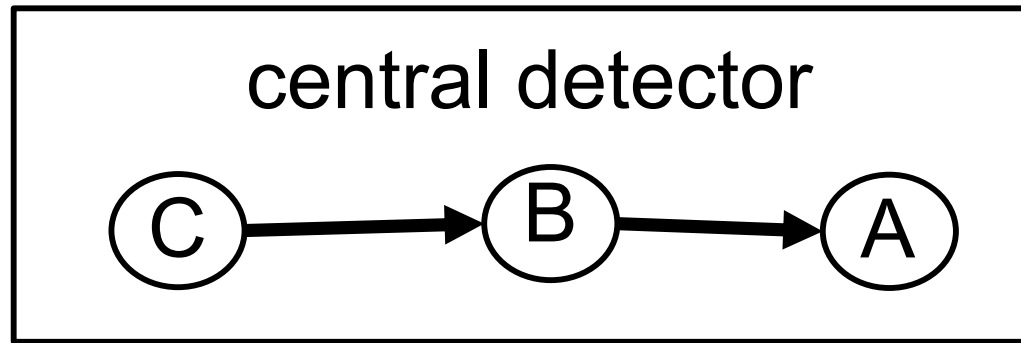
A deadlock detection algorithm must satisfy the following two conditions:

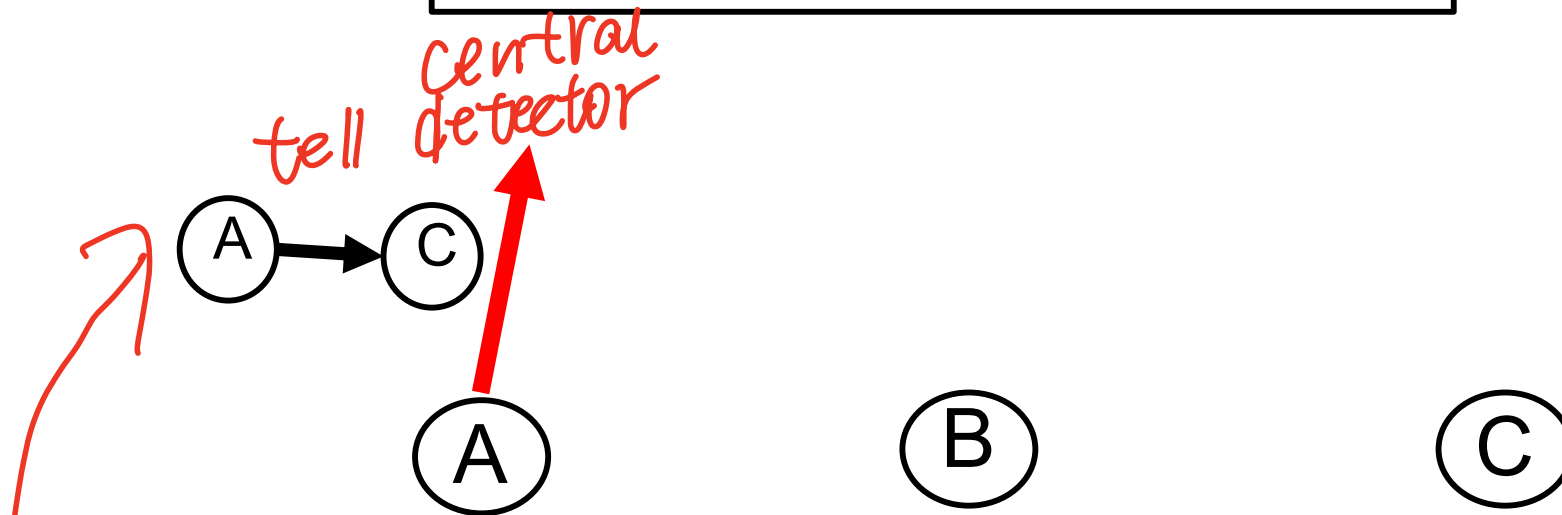
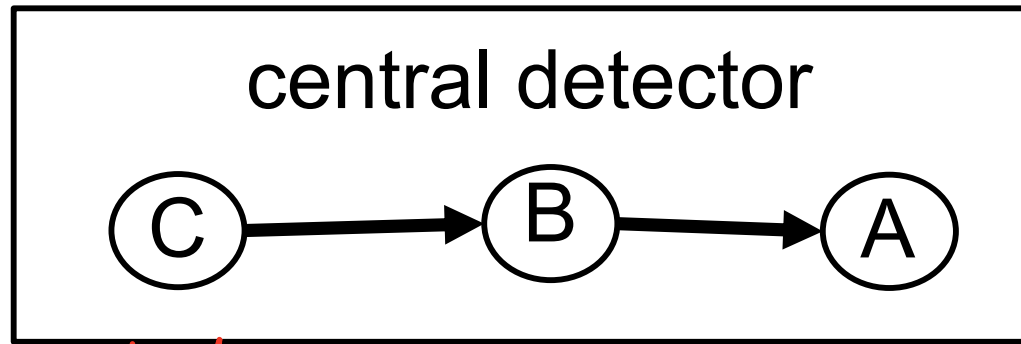
- ① ☐ the scheme does not detect a false deadlock
- ② ☐ all deadlocks will be detected

Resolution of a detected deadlock

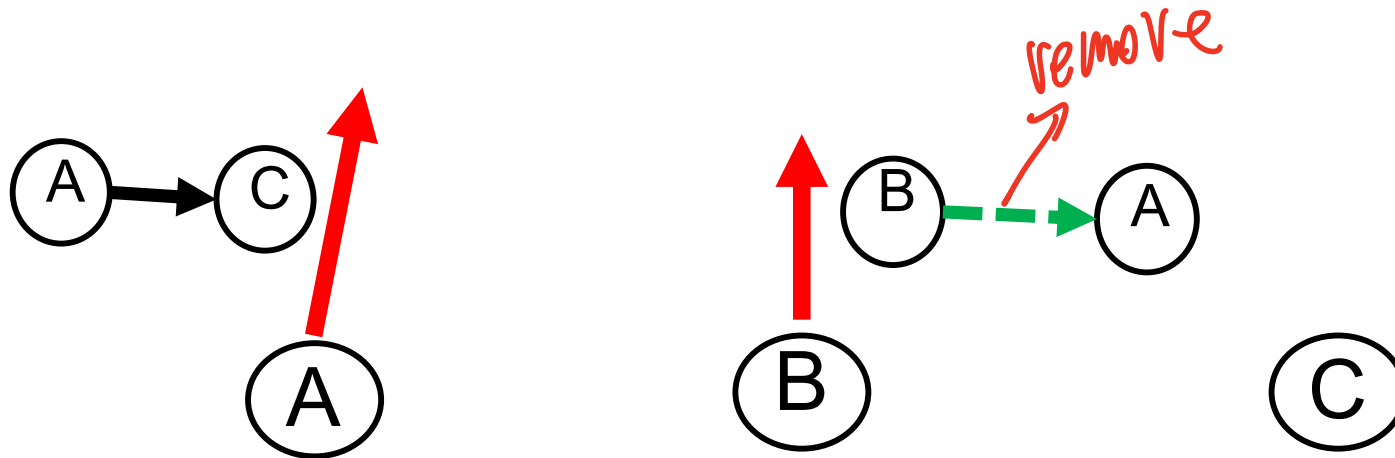
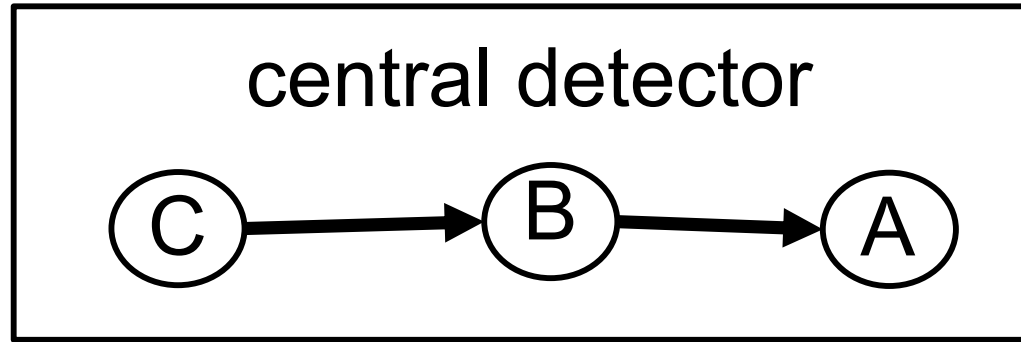
- ☐ Deadlock resolution involves breaking existing wait-for dependencies between the processes
- ☐ It involves rolling back one or several deadlocked processes and assigning their resources to blocked processes so that they can resume their execution

A centralized detection scheme

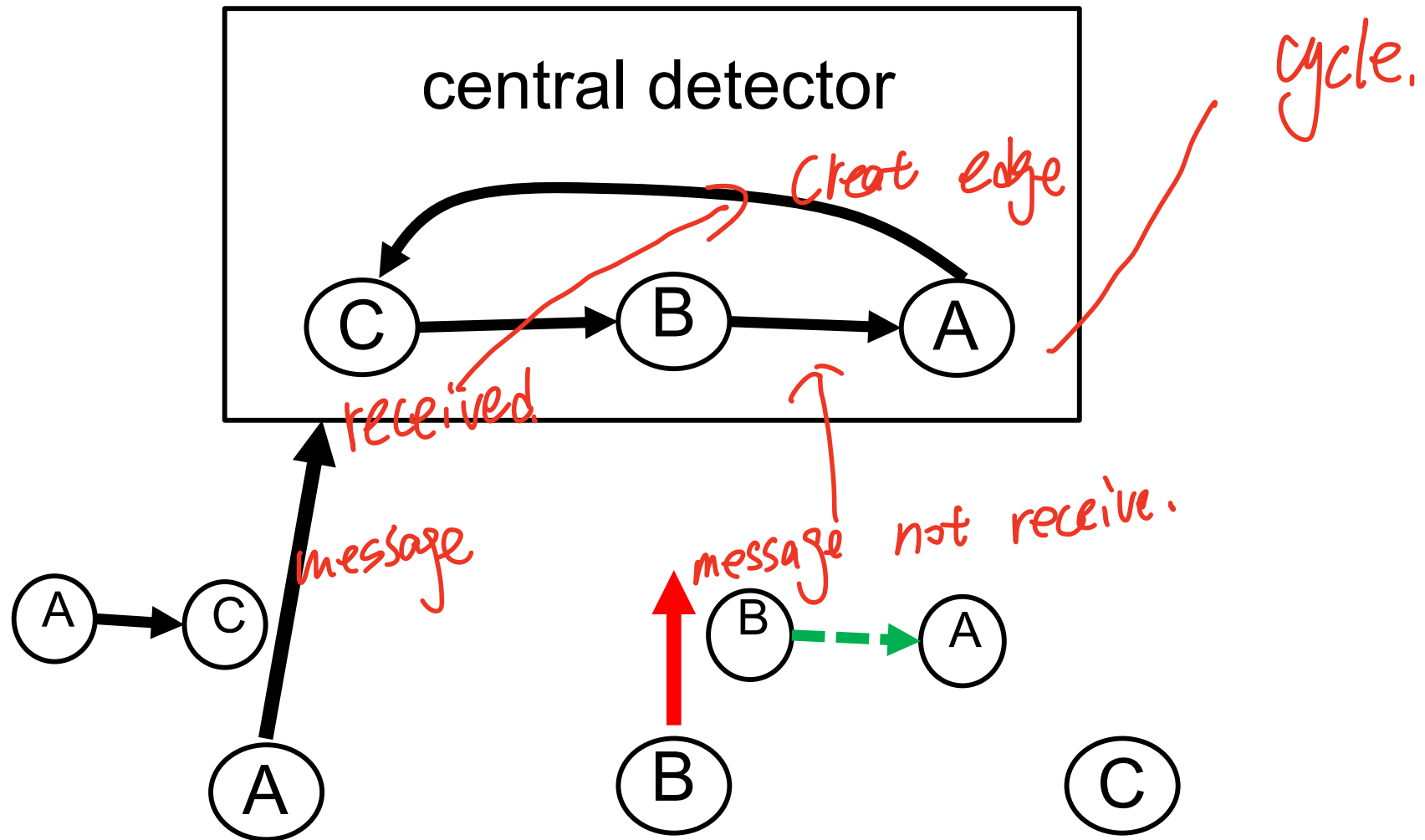




- A releases resource waited by B
- A starts waiting for resource held by C



- B can proceed, i.e. $B \rightarrow A$ should be removed



- A's message arrives first
- A false deadlock is declared

Distributed Scheme for AND model

- ❑ Designed for detecting the cycle which spans over several machines.
- ❑ The algorithm is invoked when a process starts waiting.
 - A probe (i, j, k) is generated and sent to the process (or processes) holding the needed resources.
 - i is the initiator of the probe, j is the sender, and, k is the receiver of the probe.

- ❑ When the probe is received, the recipient checks to see if itself is waiting for any processes.
 - If so, the probe is updated and sent to the processes being waited for.
 - Otherwise, discard the probe.
- ❑ If the probe comes back to the original sender, a cycle exists and there is a deadlock in the system.

Distributed Scheme for OR model

- ❑ Based on the echo algorithm
- ❑ A blocked node passes probes along the edges in the wait-for graph
- ❑ A node only executes the algorithm (i.e., passes the probes or sends the echoes) if the node is blocked
- ❑ If the algorithm terminates, then every node in the wait-for graph is blocked.
 - Deadlock

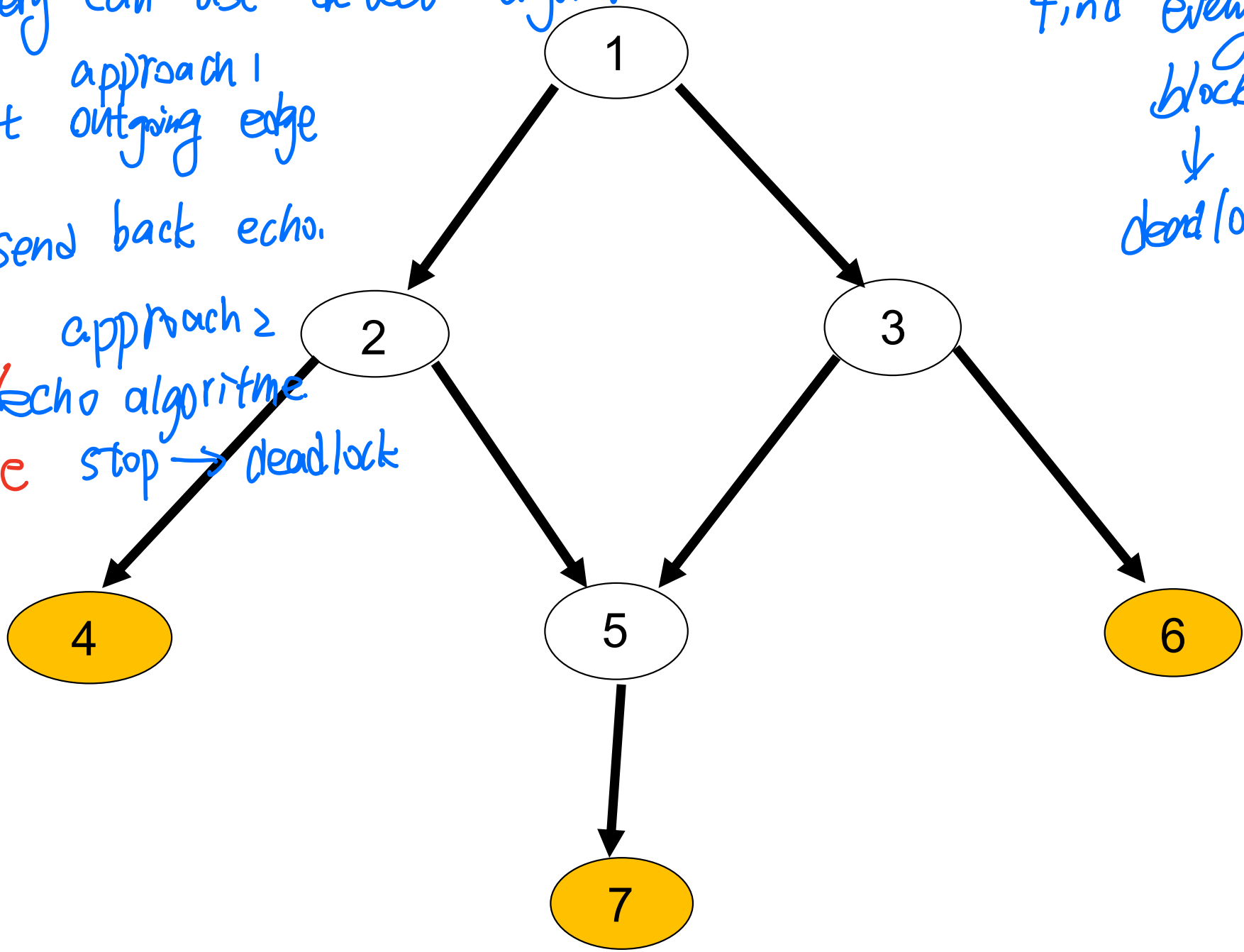
echo
↓

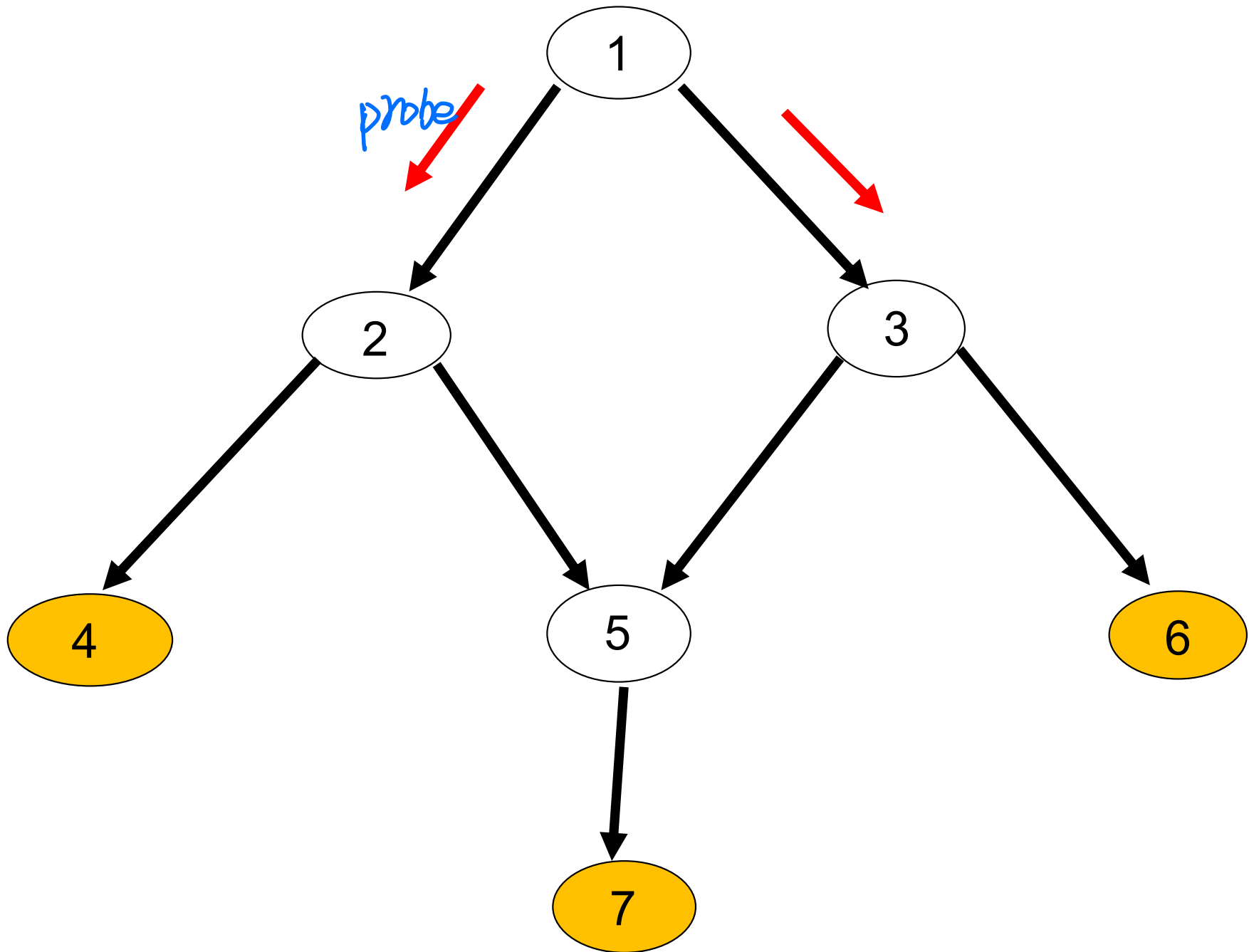
every can use detect algorithm

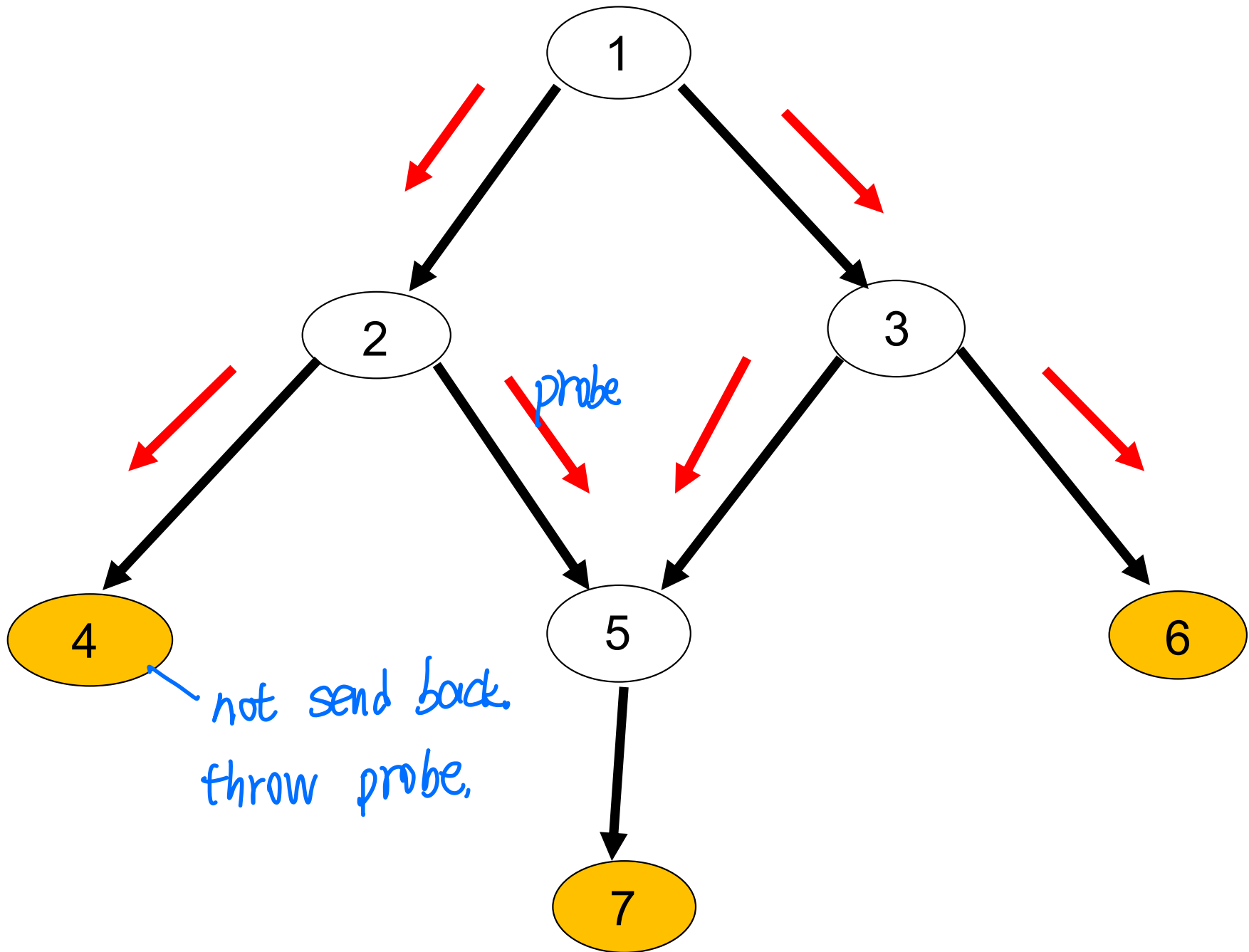
approach 1
not outgoing edge
send back echo.

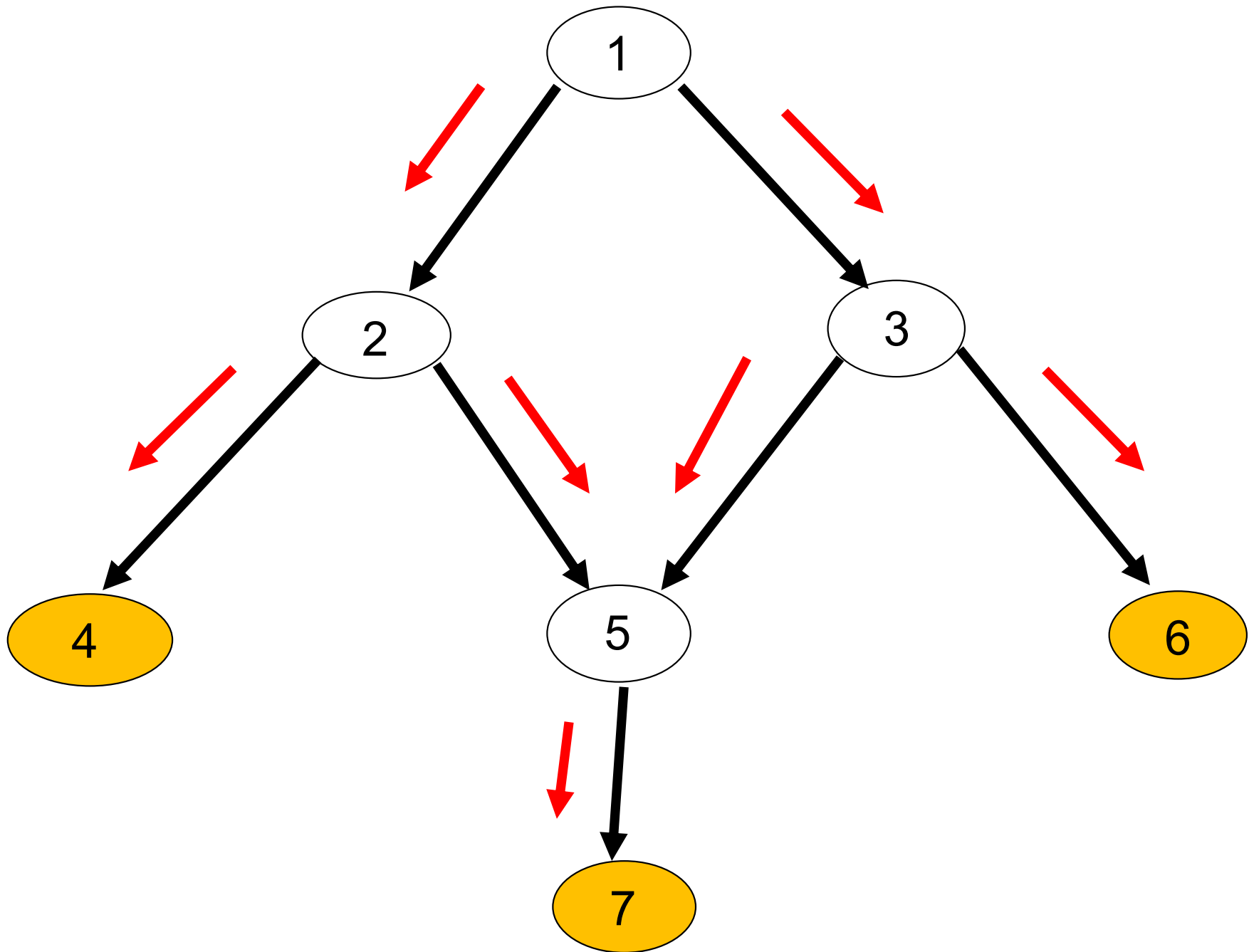
find every one
block
↓
deadlock.

approach 2
echo algorithm
use stop → deadlock

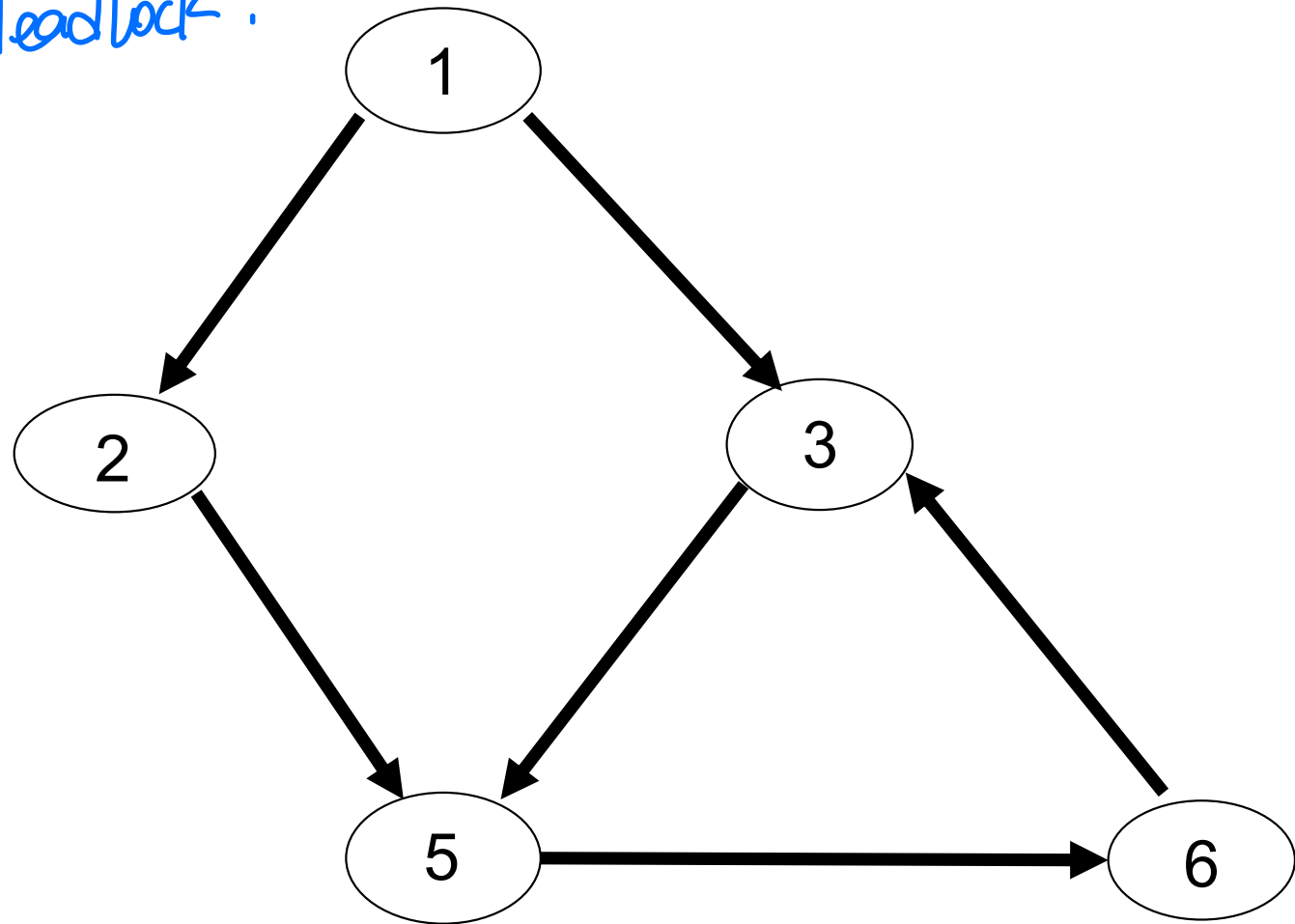


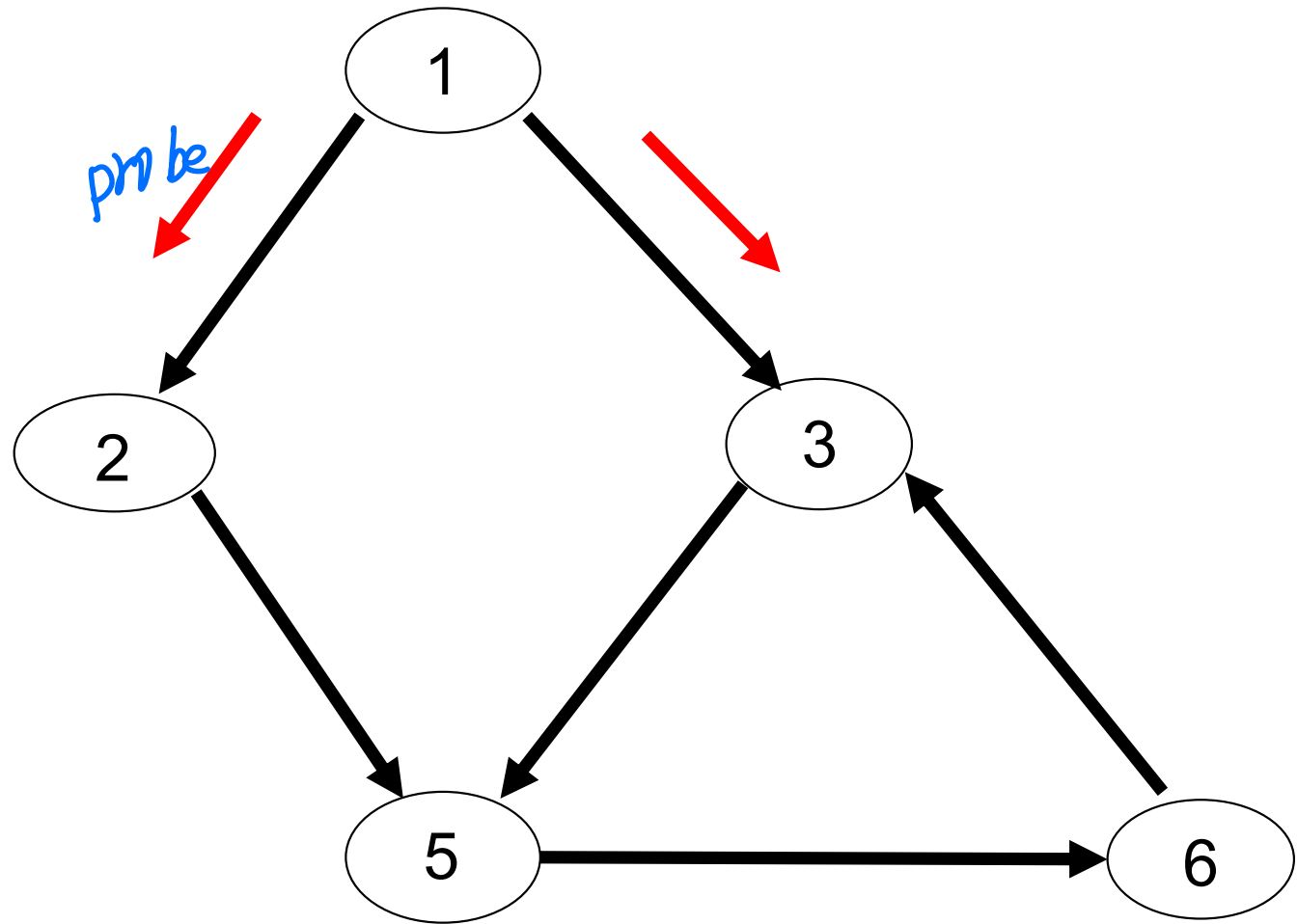


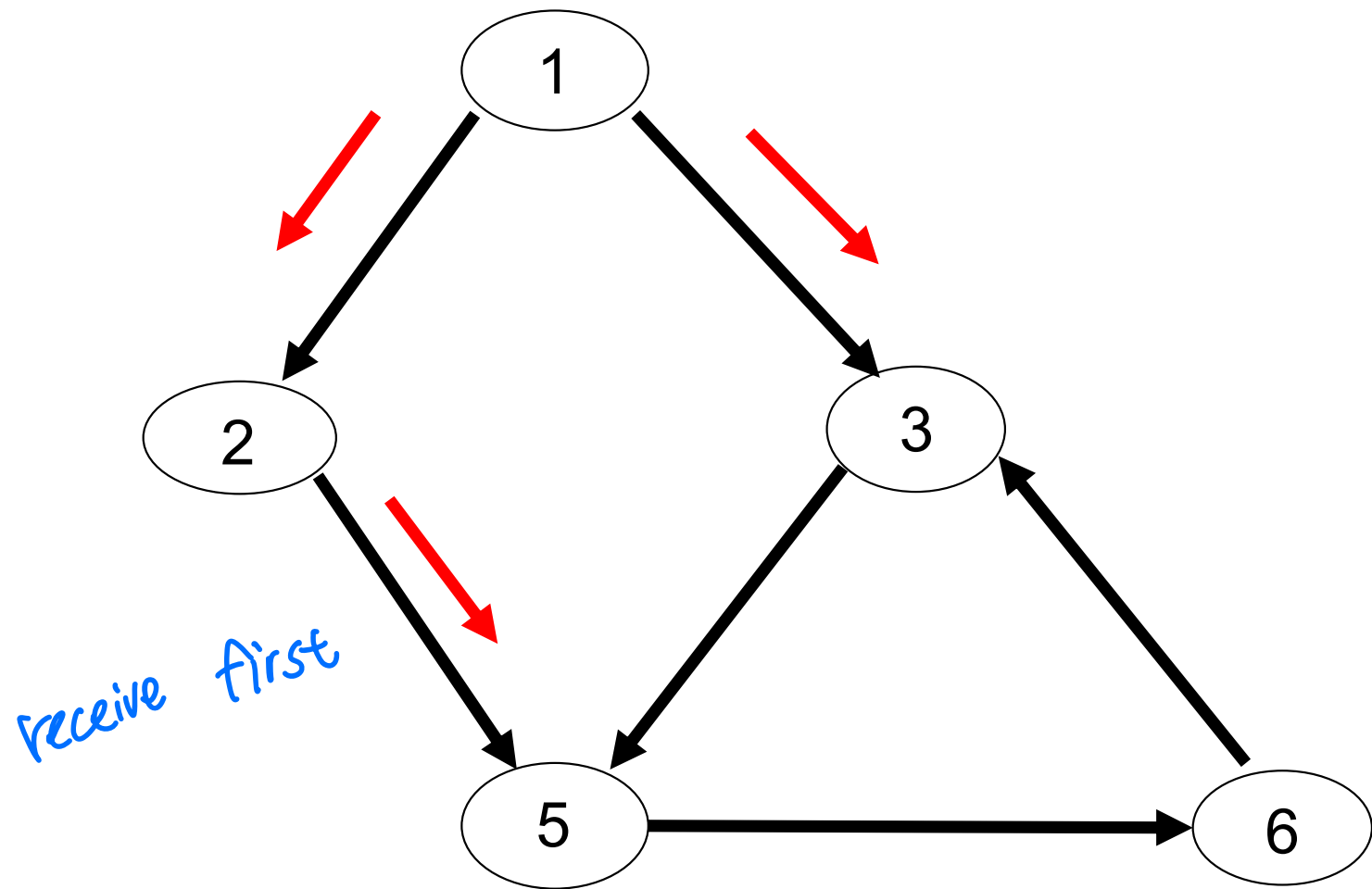


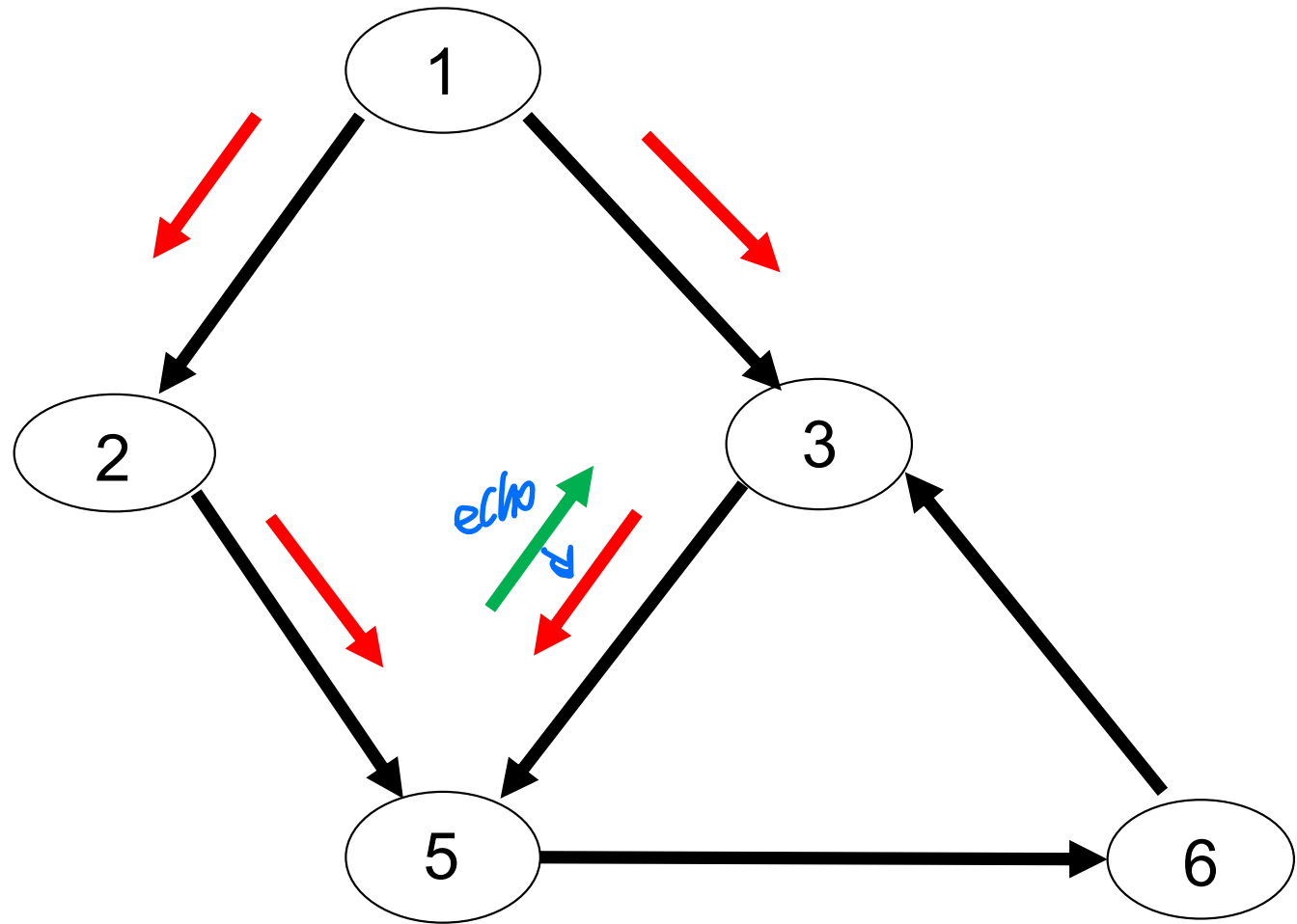


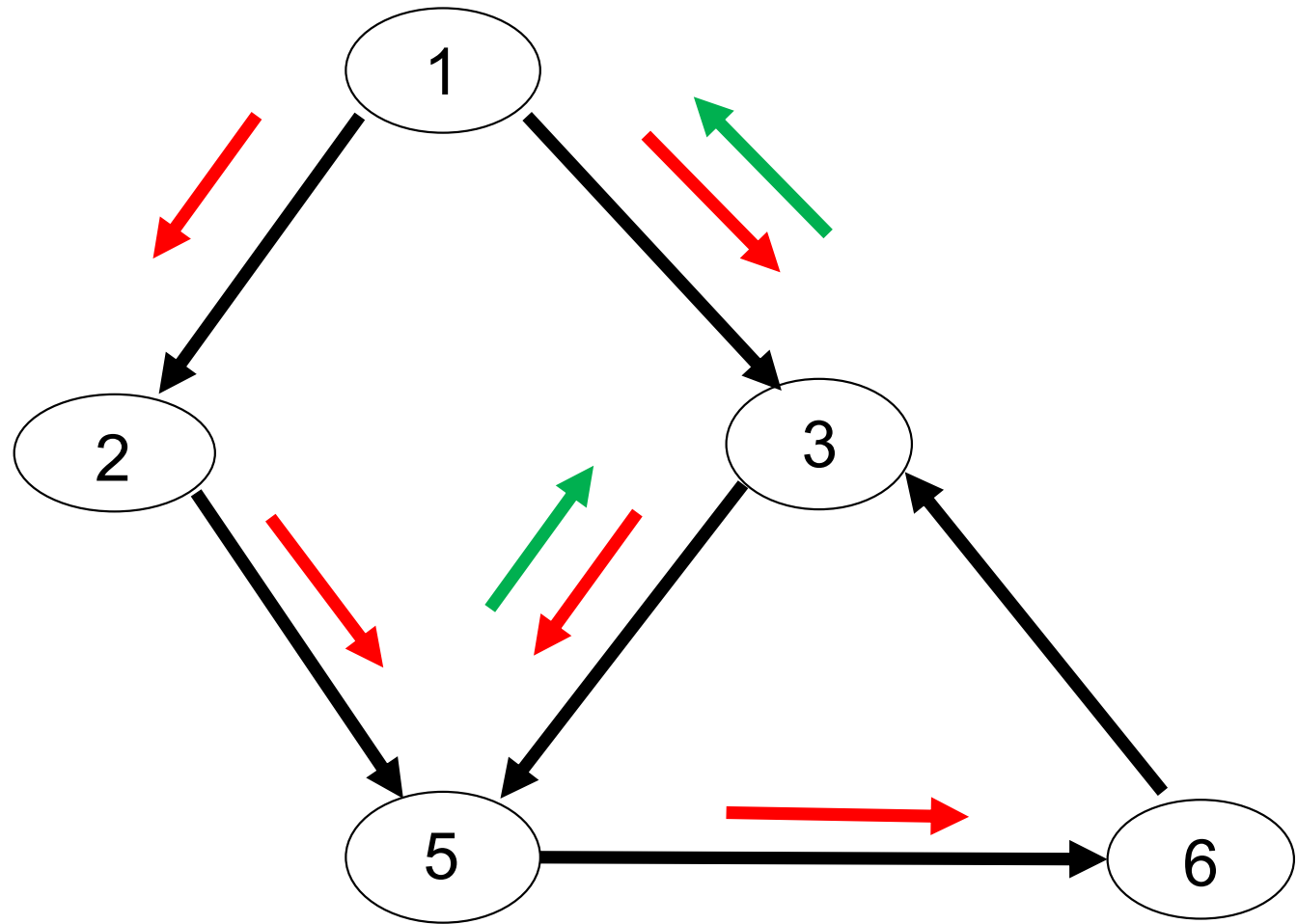
if deadlock.

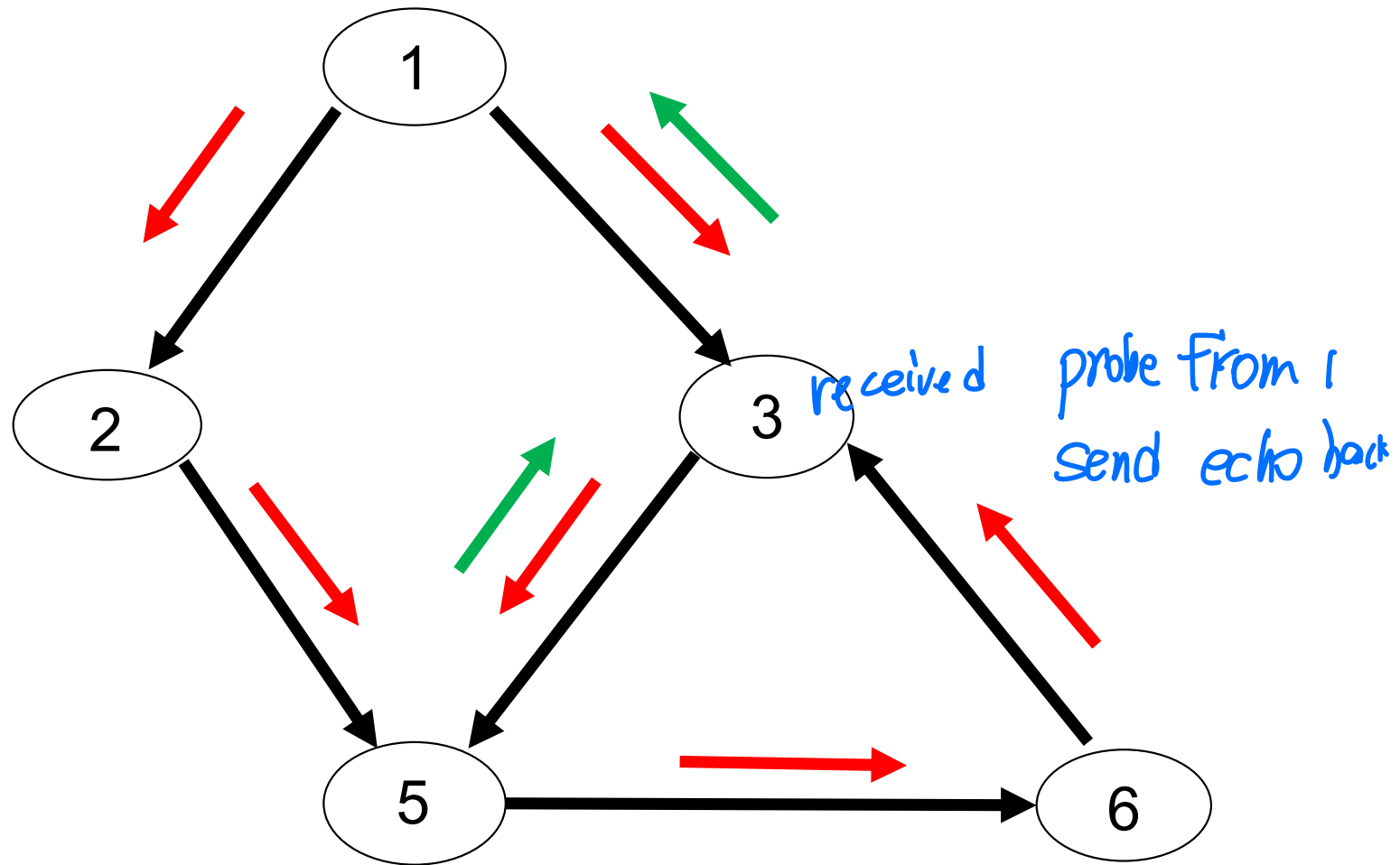


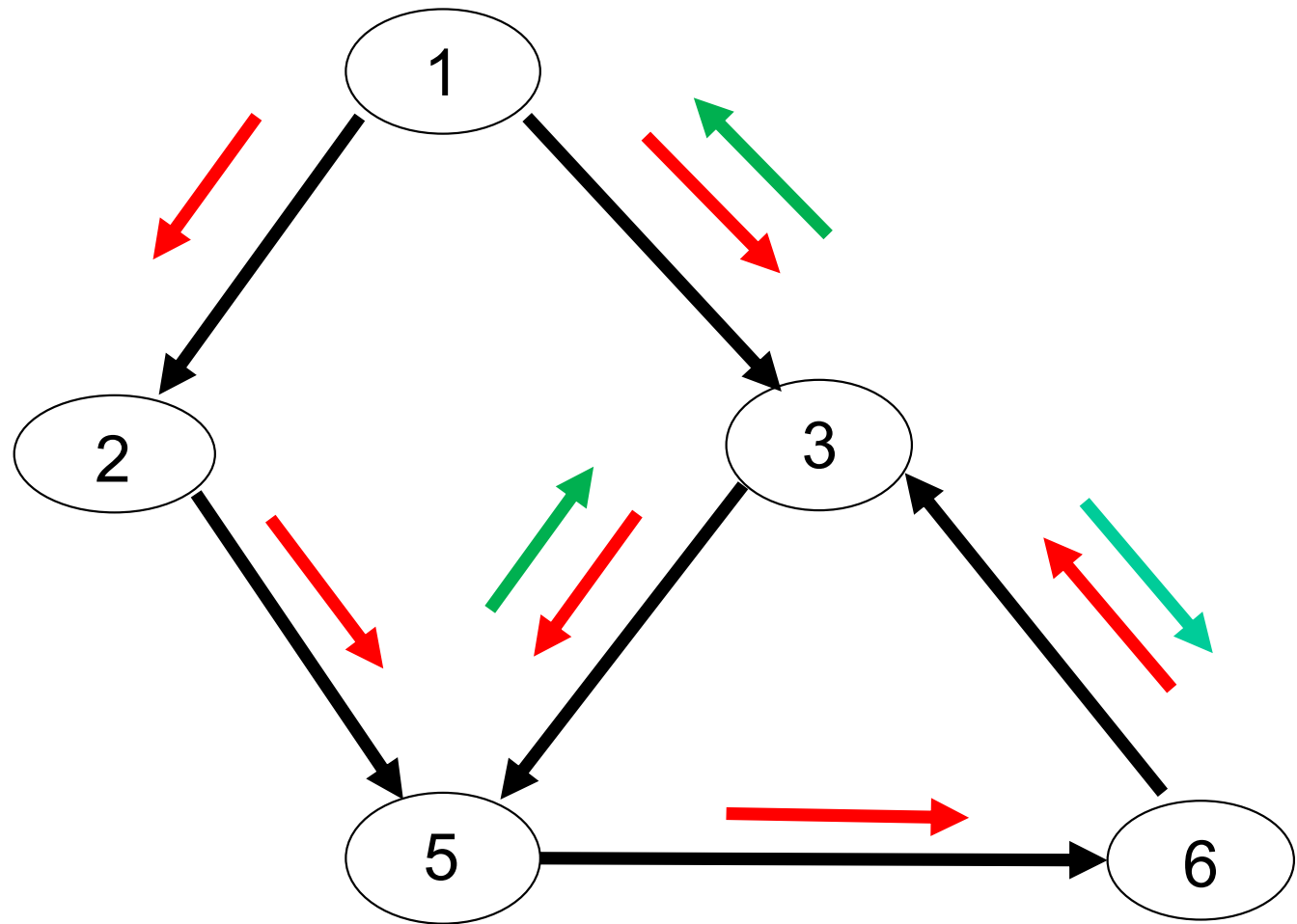


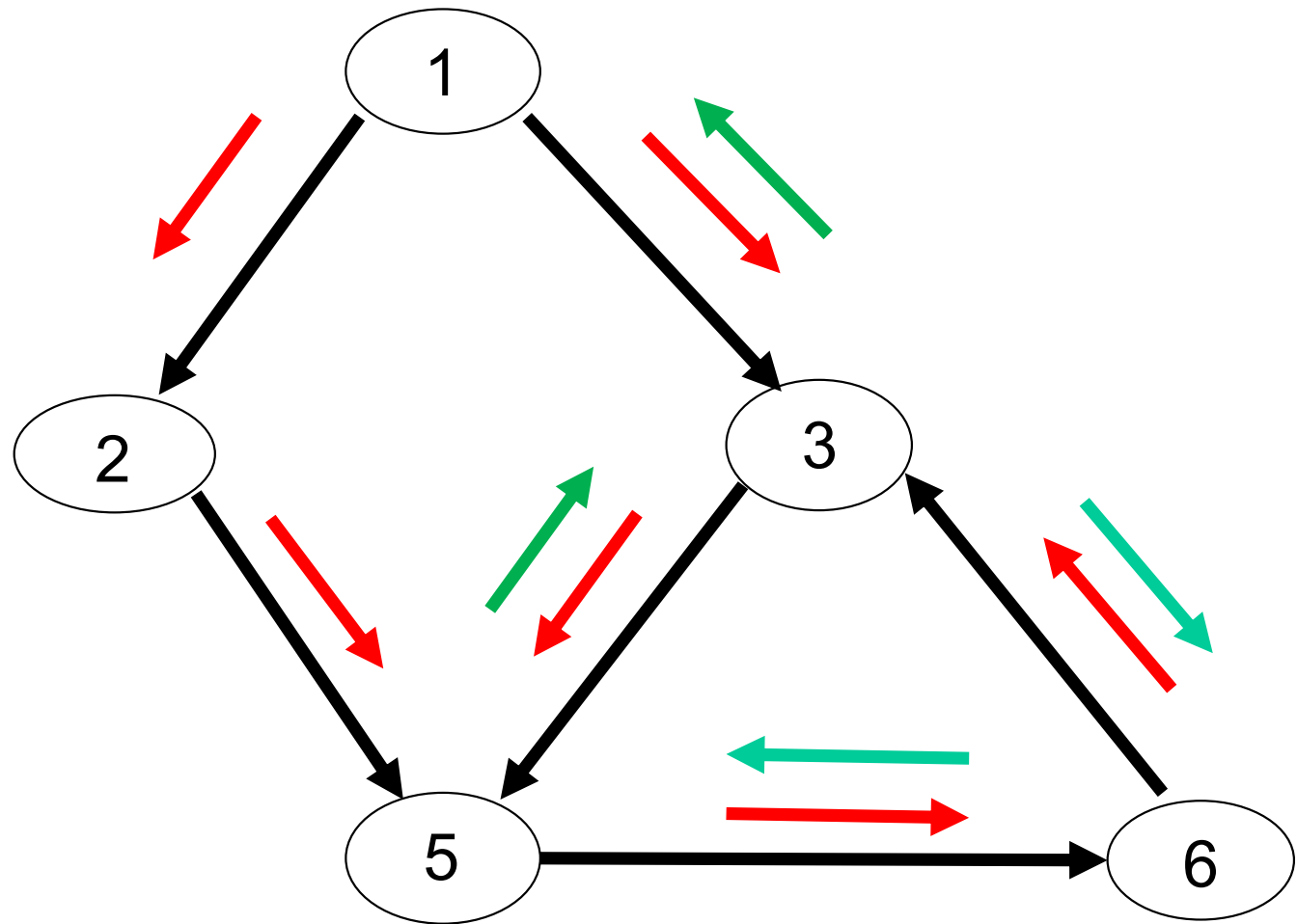


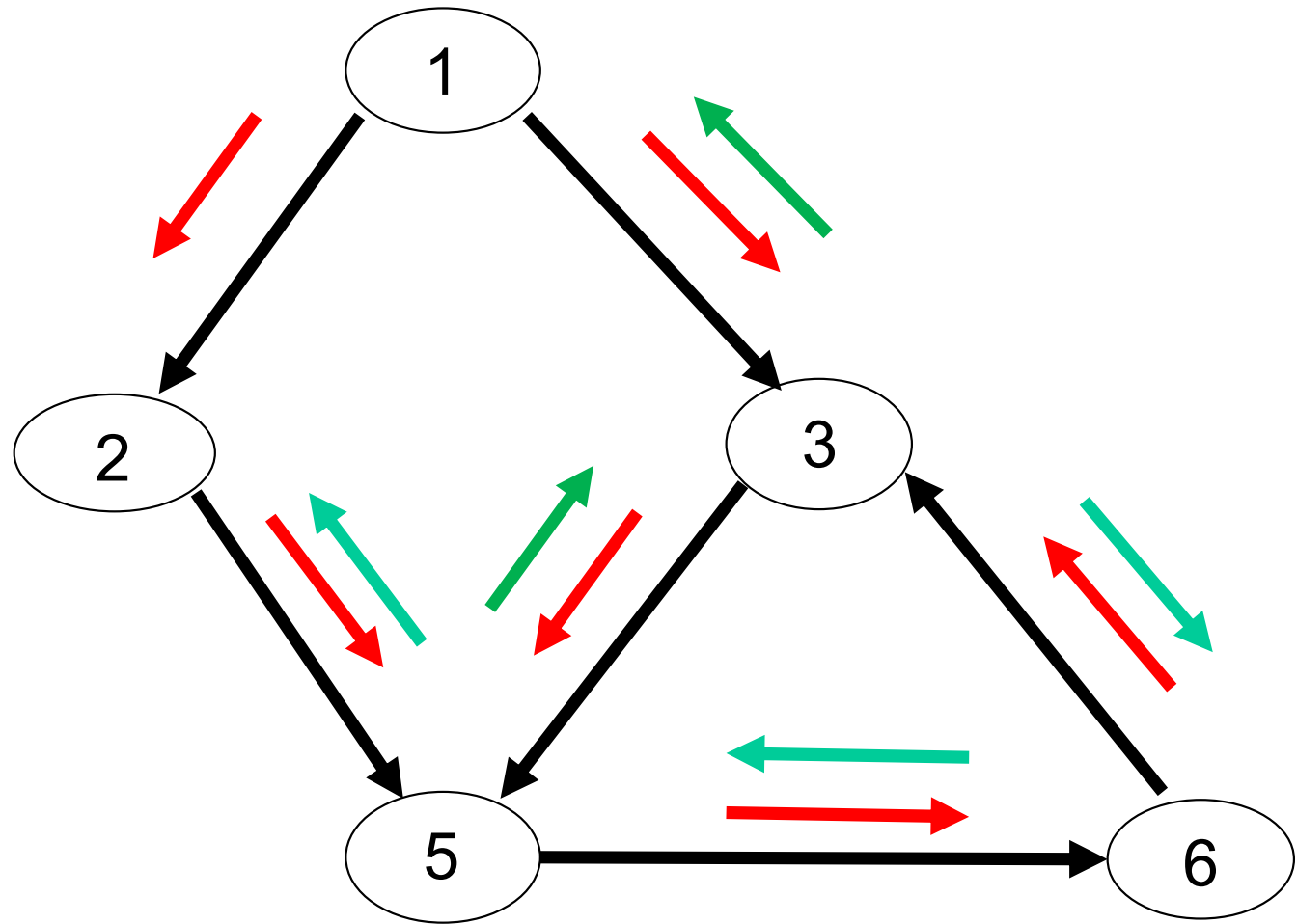


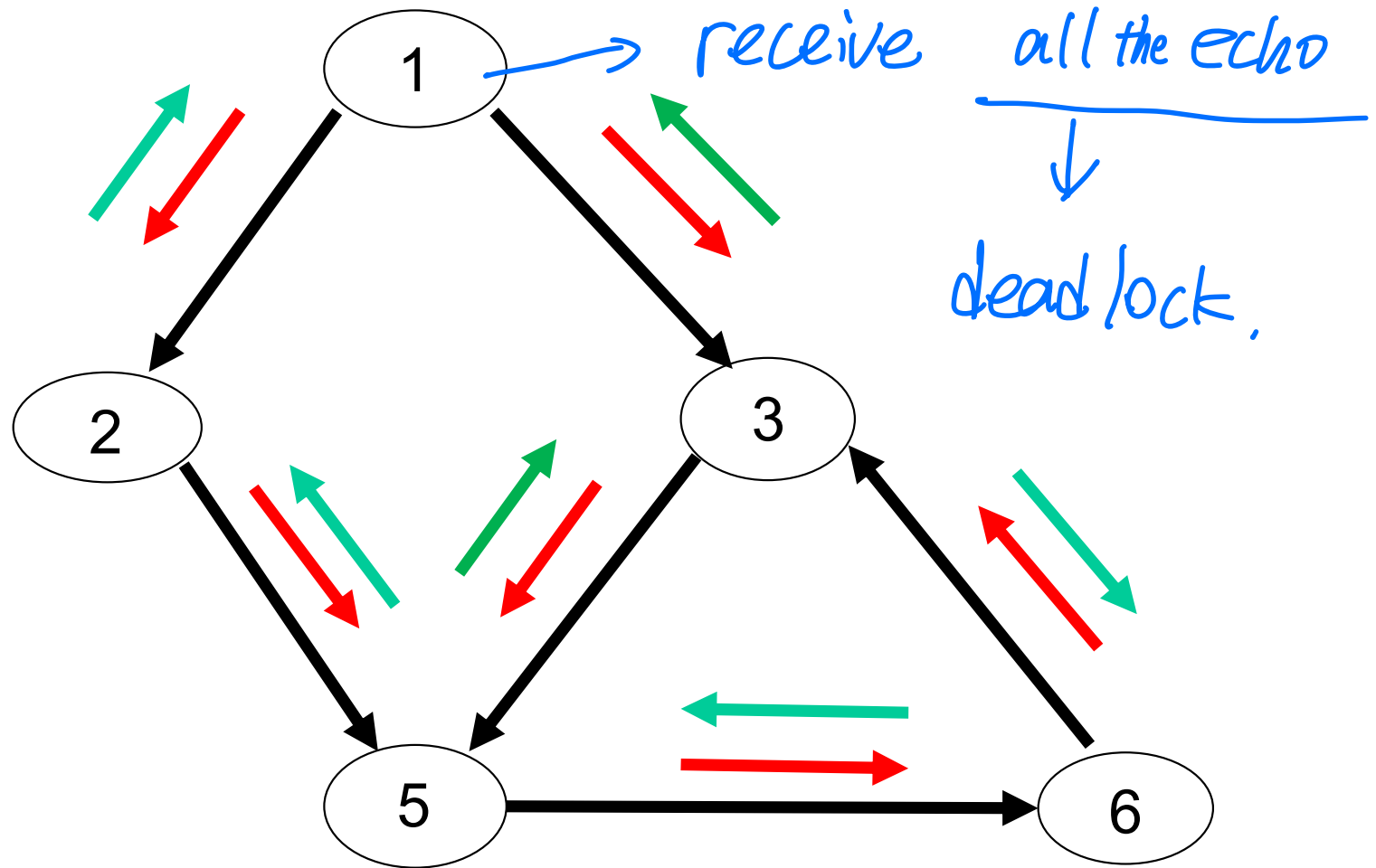












Further reading

- ❑ G. Andrews. *Paradigms for process interaction in distributed programs*. ACM Computing Surveys, 23(1):49--90, March 1991 (echo algorithm)
- ❑ K. Mani Chandy , Jayadev Misra , Laura M. Haas, Distributed deadlock detection, ACM Transactions on Computer Systems (TOCS), v.1 n.2, p.144-156, May 1983 (deadlock detection)

Review

- ❑ Describe the echo algorithm using pseudo code.
- ❑ Understand the deadlock detection schemes for the AND and OR model.
- ❑ Assume that we are running a computation that consists of several tasks. The tasks are distributed across the machines in a distributed system. Each task only knows the location of the tasks that it communicate with. Suppose we want to find out the CPU load information on each of the machines hosting the tasks. Outline an algorithm that finds the CPU load information from the machines that host the execution of the tasks.
- ❑ Suppose all the processes in the system has a distinct ID that can be used to uniquely identify the processes in the system. Modify Chandy's deadlock detection for the AND model so that, when a deadlock is detected, the process with the smallest ID can be aborted to resolve the deadlock.